Smart Medicine Reminder & Expiry Tracker System

Project Report

Project Members

Faaeq Sayed (24101A0010)

Advay Tongaonkar (24101A0073)

Vidyalankar Institute of Technology

June 2026

1. Project Overview

The Serverless Smart Medicine Reminder & Expiry Tracker System is a cloud-based healthcare utility designed to make sure patients never miss a dose and are always warned before their medicines expire. Think of it as a personal, automated pharmacy assistant running 24 hours a day, 7 days a week, entirely in the cloud, with no need for any server the team has to maintain themselves.

The system works in a simple way: a patient visits a clean, modern web page and enters their details once heir name, email address, medicine name, dosage, the time they need to take it, and the expiry date. After that, the system takes over completely. Every day, at exactly the right moment, it sends the patient a reminder email. And every morning at 9:00 AM, it checks whether any medicine is expiring soon or has already expired, and sends out a warning email automatically.

The key idea behind the project is automation and reliability. Once registered, the patient does not need to touch anything. The cloud infrastructure handles all reminders, warnings, and record-keeping without any manual effort.

---

2. The Problem This Project Solves

Managing medicines manually is a real challenge for many patients, especially the elderly or those juggling multiple prescriptions. Common problems include:

- Forgetting to take medicines at the scheduled time.
- Accidentally consuming medicines that have already expired.
- Not realising a medicine is about to expire until it's too late to get a replacement.

Traditional reminder apps require the patient to interact with the app every day, set manual alarms, or rely on physical pill organisers. This project eliminates all of that by making the cloud do the work the patient just needs access to their email inbox.

---

3. System Architecture → How Everything Connects

The project is built on Microsoft Azure, which is one of the world's largest cloud computing platforms. The architecture is described as "serverless," which simply means the team did not have to set up, manage, or pay for a dedicated server. Instead, small pieces of code (called functions) run only when needed, making the system extremely cost-efficient.

Here is a plain-English walkthrough of how the four main components connect to each other:

3.1 Frontend Web Interface (The Form)

The patient opens a web browser and visits the Medicine Reminder System page. The interface is built using standard web technologies HTML, CSS, and JavaScript. It has a modern, dark-mode design with a clean glassmorphic look (blurred glass effect). The patient fills in their details and clicks "Add Medicine." That click sends all the data securely to the cloud.

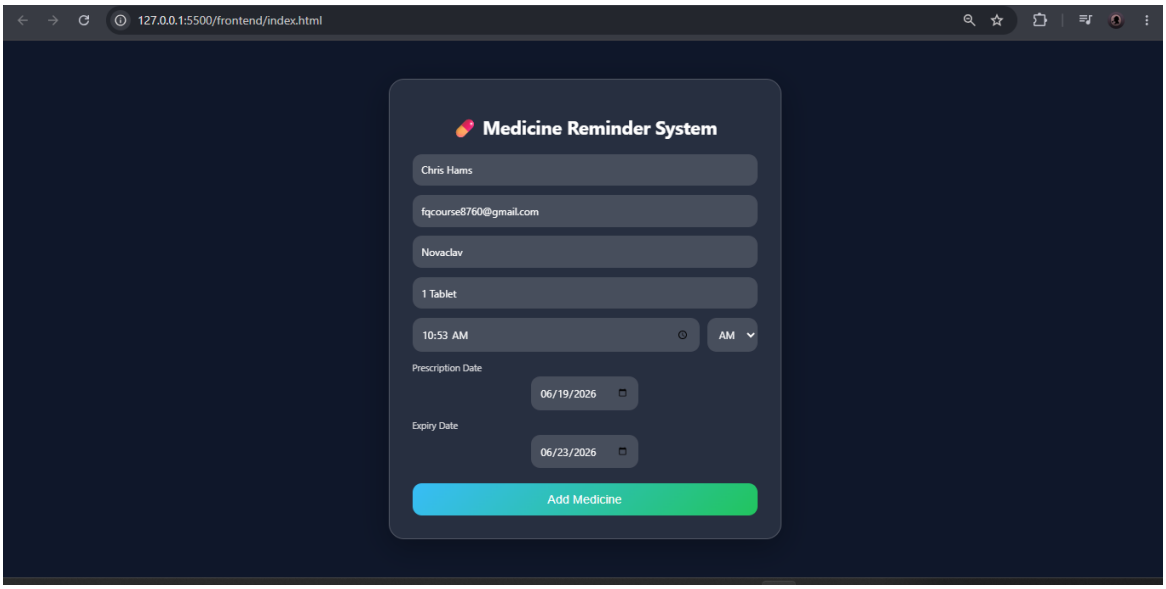


Figure 1: The Medicine Reminder System web form where patients enter their details

3.2 Azure Blob Storage → The Database (medicines.json)

Rather than using a heavy, expensive database like SQL Server, this project stores all patient records in a simple JSON file called medicines.json, hosted inside Azure Blob Storage. Blob Storage is essentially a cloud filing cabinet fast, cheap, and always available.

Each record in the JSON file stores: the patient's name, email, medicine name, dosage, scheduled time, prescription date, expiry date, and three important "lock" fields that prevent duplicate emails from being sent (explained in Section 5).

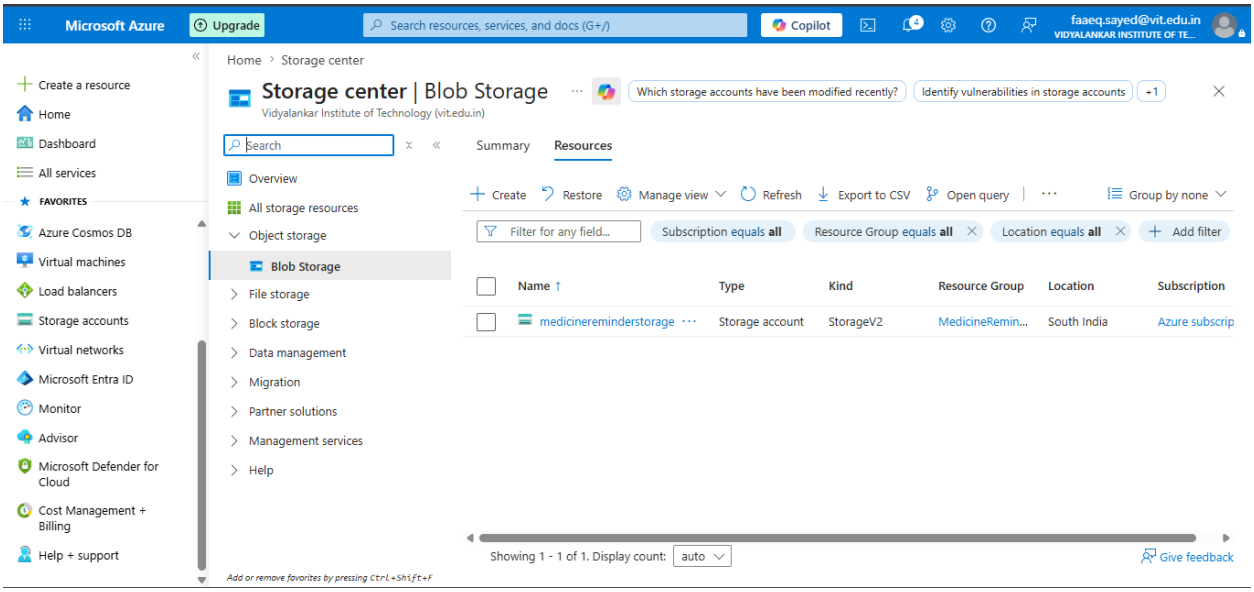


Figure 2: Azure Blob Storage account (medicinereminderstorage) containing the prescriptions container

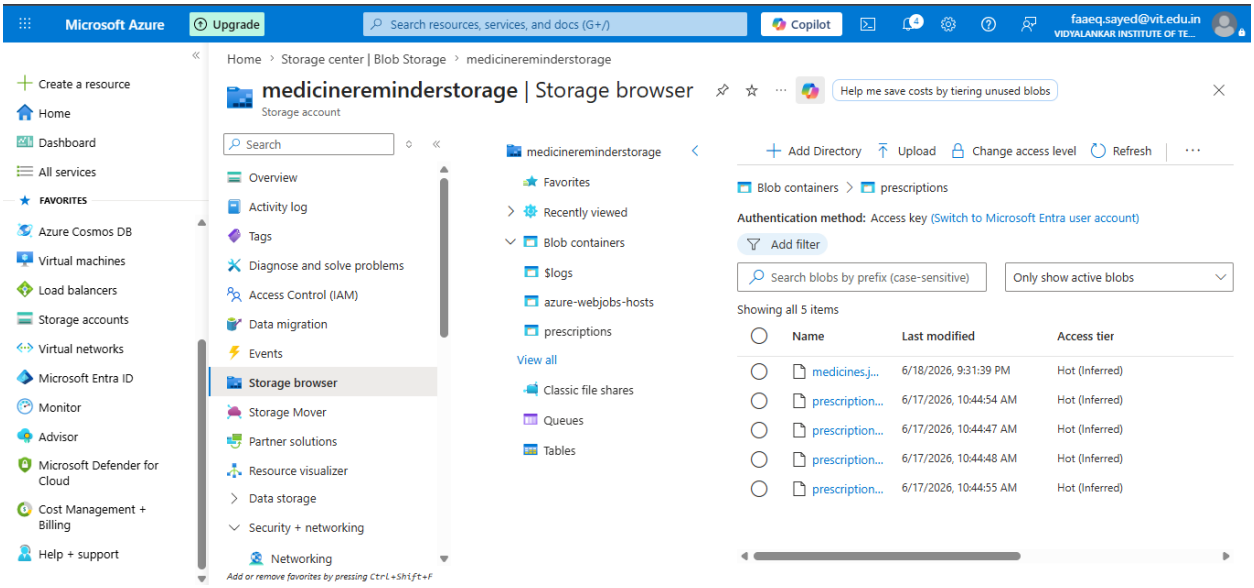


Figure 3: The storage browser showing the medicines.json file and related blobs

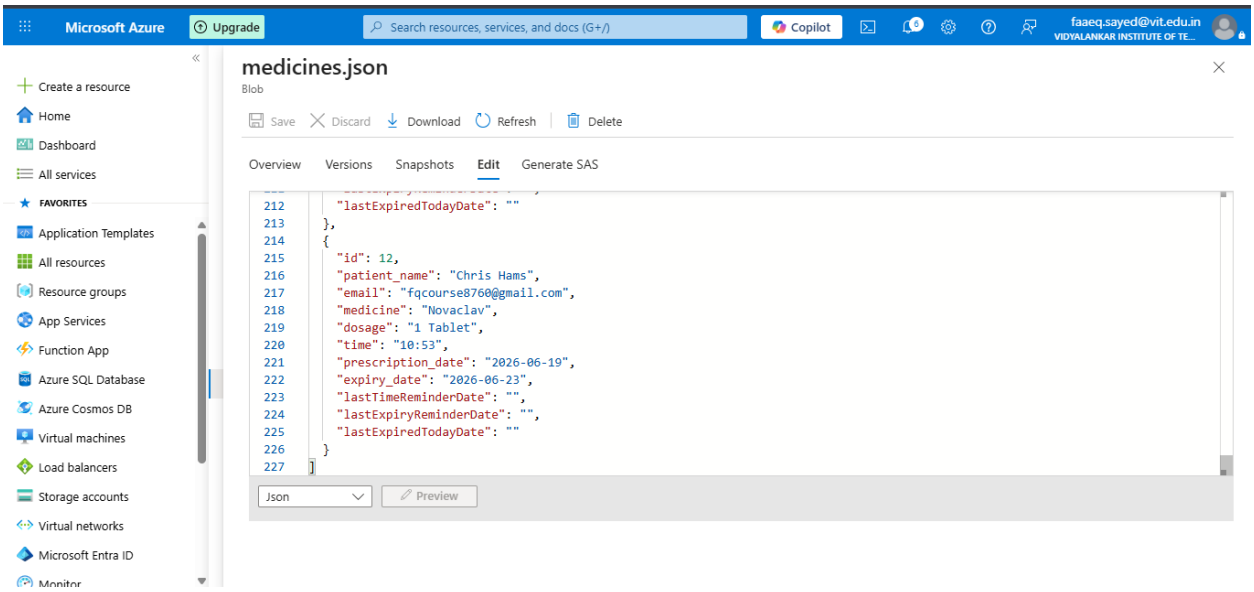


Figure 4: A live view of the medicines.json file inside Azure — showing a patient record for Chris Hams taking Novaclav

3.3 Azure Functions → The Brain (Two Microservices)

Azure Functions are small, event-driven programs that run in the cloud only when triggered. This project uses two functions:

AddMedicine Function: When the patient submits the form, this function receives the data, saves the new patient record into the JSON file in Blob Storage, and immediately sends a "Medicine Added" confirmation email to the patient. It also initialises the three date-lock fields to empty strings so the system knows this patient's reminders have not yet been sent.

CheckMedicineReminders Function: This is the heart of the automation. Every minute, it wakes up and reads every record in the JSON file. It checks: (1) Has this patient's scheduled medicine time arrived right now? If yes, send a reminder email. (2) Is it 9:00 AM? If yes, check whether any medicine is expiring tomorrow, or has expired today, and send the appropriate warning.

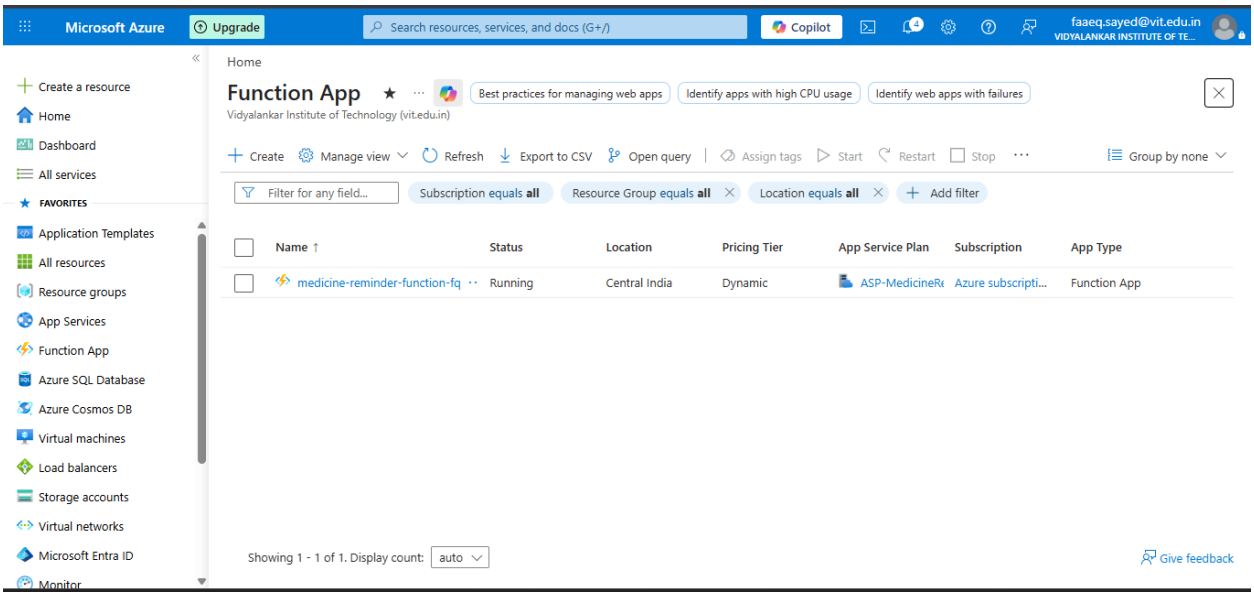


Figure 5: The Azure Function App (medicine-reminder-function-fq) running on the Dynamic (Consumption) pricing plan

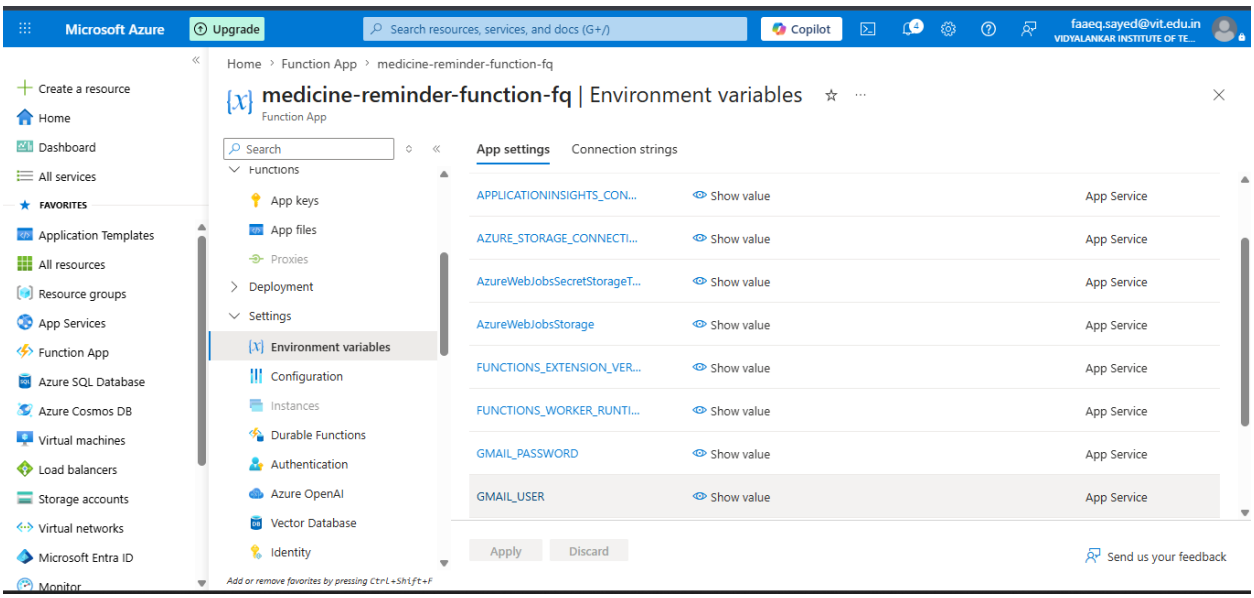


Figure 6: Environment variables configured in the Function App — including Gmail credentials and Azure Storage connection strings

3.4 Azure Logic App → The Scheduler

This component is the most important piece of the project. Azure Functions on the Consumption plan "sleep" when there is no activity, to save costs. If the function is asleep at 9:00 AM when it should be sending expiry warnings, those emails would never go out.

The Logic App solves this by acting as a scheduled alarm clock. It wakes up every day on a fixed schedule (set to India Standard Time) and sends an HTTP POST request directly to the Azure Function URL. This "poke" wakes the function up and forces it to run its check immediately, ensuring no reminder is ever missed.

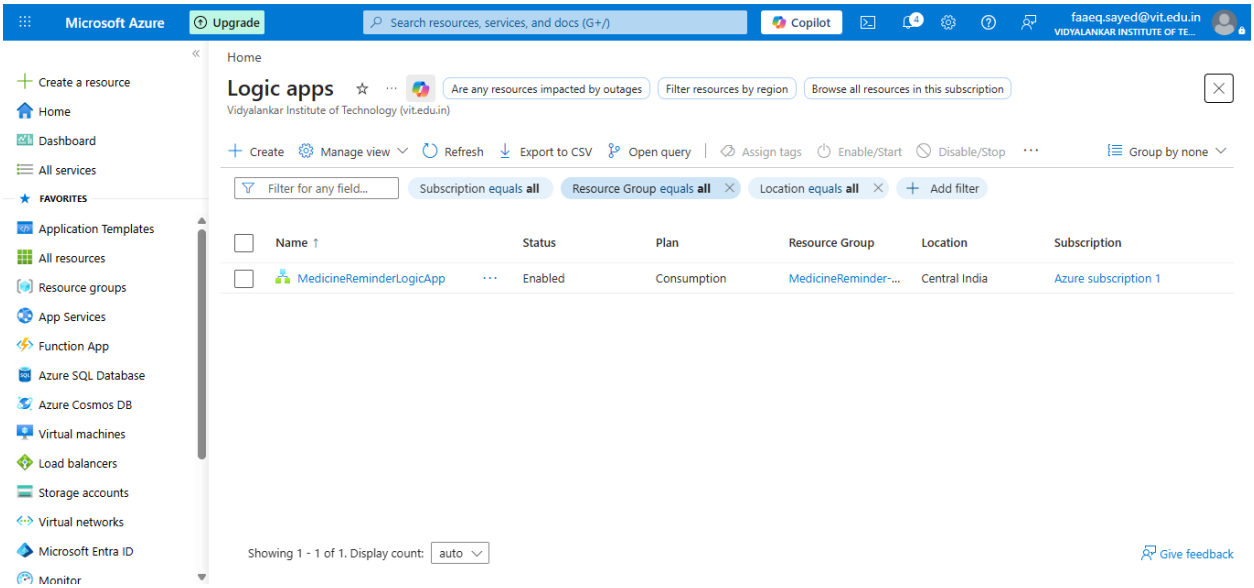


Figure 7: The MedicineReminderLogicApp listed as Enabled under the Consumption plan in Central India

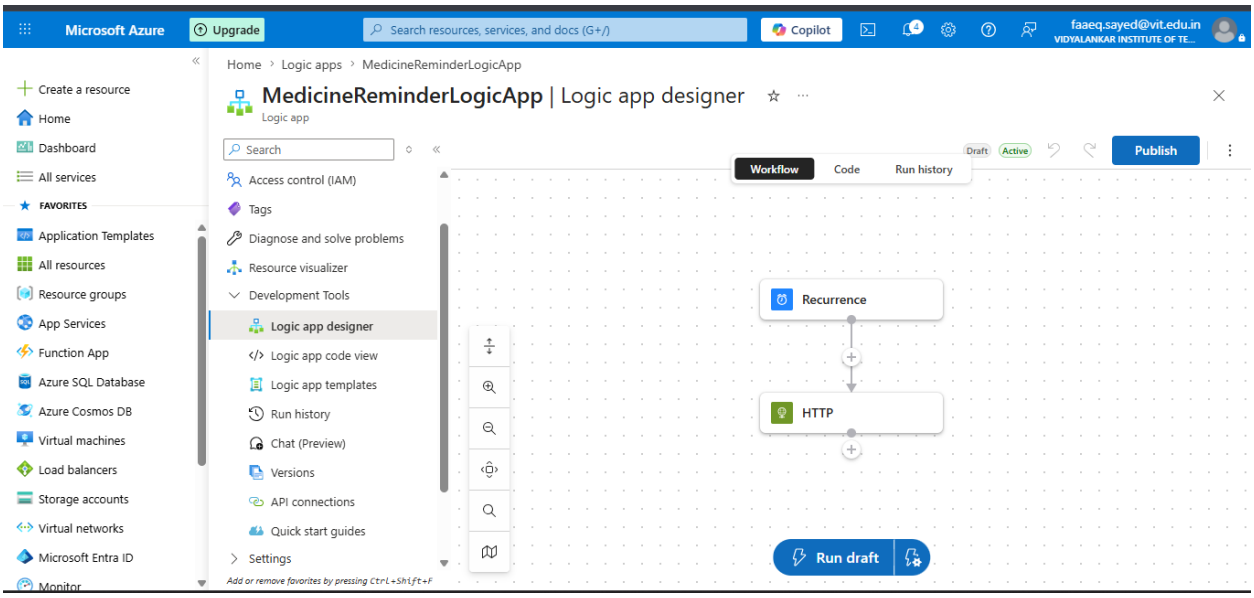


Figure 8: The Logic App designer showing the simple two-step workflow — Recurrence trigger followed by an HTTP action

4. Technology Choices → Why These Tools?

4.1 Why Microsoft Azure?

Azure was chosen because it offers a generous free trial for students, and the team had access to the Azure for Students subscription through Vidyalankar Institute of Technology. Azure also provides a complete ecosystem storage, functions, logic apps, and monitoring all in one place, with a single dashboard to manage everything.

4.2 Why Node.js (JavaScript) for the Backend?

This is an important technical decision worth explaining clearly. Azure Functions support multiple programming languages Python, C#, Java, and Node.js (JavaScript). However, the available programming language is directly tied to the hosting plan selected.

The team chose the Consumption Plan for hosting the Azure Functions. The Consumption Plan is the most budget-friendly option you only pay for the exact time the function is actually running. For a student project with low traffic, this is ideal and falls well within the free tier.

The alternative, the Flex Consumption Plan, offers more features and faster cold starts, but it does NOT support free trial or student accounts. The team attempted to use the Flex Consumption Plan but encountered compatibility issues with the free Azure subscription. The standard Consumption Plan was the only viable option.

The Consumption Plan natively supports Node.js (v20 runtime), and JavaScript was already familiar to the team from the frontend work. Choosing Node.js meant the same language JavaScript could be used consistently across both the frontend web page and the backend cloud functions. This reduced context-switching, simplified debugging, and allowed code patterns (like JSON parsing) to be shared easily across the project.

4.3 Why Nodemailer + Gmail SMTP?

Nodemailer is a widely-used Node.js library for sending emails. It was configured to route emails through a Gmail account using Google's SMTP server. This is a zero-cost, reliable solution for a project of this scale. The Gmail credentials (email address and app password) are stored securely as environment variables inside the Azure Function App settings — never hardcoded into the source code.

4.4 Why Azure Blob Storage instead of a Database?

Setting up and maintaining a full relational database (like Azure SQL) adds cost and complexity. For this project, the data model is simple — a flat list of patient records. A JSON file stored in Blob Storage is perfectly sufficient, extremely fast to read and write, and costs a fraction of a penny per month. The function reads the entire file, makes changes in memory, and writes it back — a pattern that works beautifully at this scale.

5. The Anti-Duplicate Email System

One of the cleverest parts of this project is how it prevents sending duplicate emails. Since the CheckMedicineReminders function runs every minute (triggered by the Logic App), there is a risk that the same reminder could fire multiple times on the same day.

A naive approach would store a simple true/false flag. But flags can get stuck if the system restarts or the file write fails. Instead, this system stamps the actual calendar date as a string (for example, "2026-06-19") into three special fields in every patient record:

- lastTimeReminderDate — the date the medicine dose reminder was last sent.
- lastExpiryReminderDate — the date the expiry warning was last sent.
- lastExpiredTodayDate — the date the "expired today" alert was last sent.

Every time the function runs, it compares today's date to these stored date strings. If they match, the email is skipped. If they don't match (because it's a new day), the email fires and the date is updated. This is a far more reliable pattern than a boolean flag — it naturally resets every midnight without any extra logic.

---

6. Development Workflow → From Laptop to the Cloud

The project was developed locally on a Windows machine using the Command Prompt (CMD), the Azure CLI, and the Azure Functions Core Tools. Here is the step-by-step process the team followed:

Step 1 — Verify the Azure Environment

```
az --version
```

This confirms the Azure command-line tools are installed and ready.

Step 2 — Log In to the Azure Account

```
az login
```

This opens a browser to authenticate the developer's Microsoft account and link it to the Azure subscription.

Step 3 — Create the Local Project Folder

```
func init Medicine_Reminder_System --javascript
```

This command scaffolds (creates the skeleton of) a new Azure Functions project configured for Node.js/JavaScript.

Step 4 — Generate the Two Function Endpoints

```
func new --name addmedicine --template "HTTP trigger" --authlevel "anonymous"
func new --name checkMedicineReminders --template "Timer trigger"
```

The first command creates the AddMedicine HTTP endpoint. The second creates the CheckMedicineReminders timer function.

Step 5 — Local Testing

```
func start
```

This runs the entire function app locally on the developer's machine at 127.0.0.1:5500, making it possible to test every feature before deploying to the cloud.

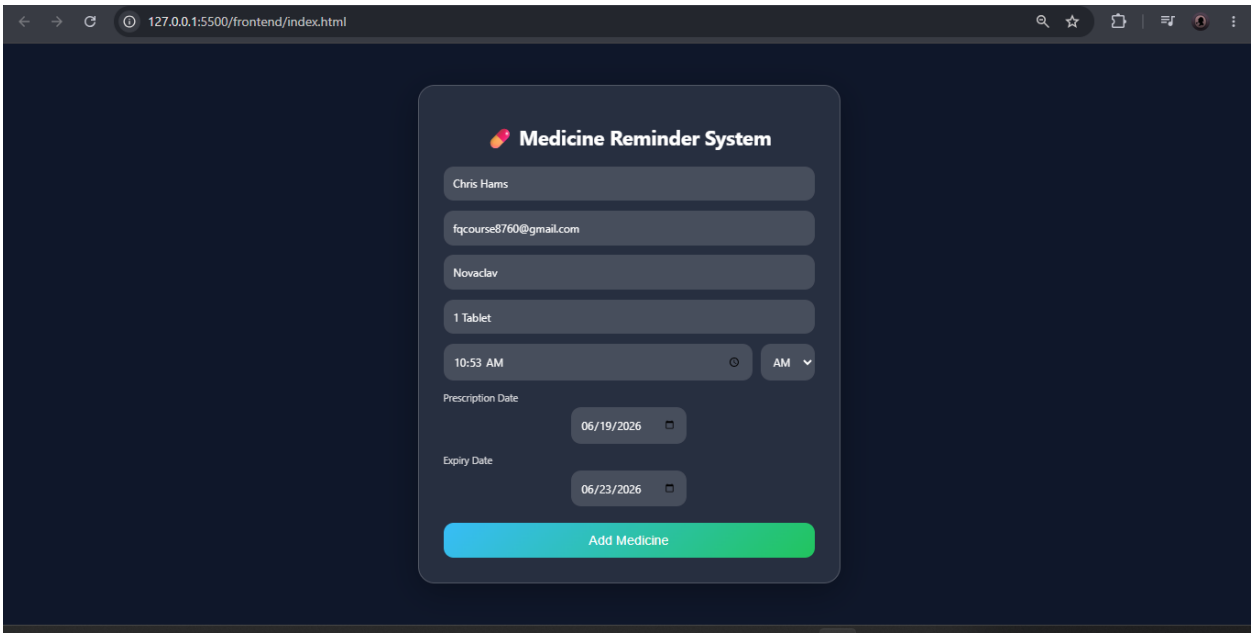


Figure 9: The web form running locally on the developer's machine, showing a successful "Medicine added" popup

Step 6 — Deploy to the Cloud

```
func azure functionapp publish medicine-reminder-function-fq
```

This single command packages and publishes the entire project to the live Azure Function App, making it accessible from anywhere in the world.

---

7. Azure Resource Group → All Components Under One Roof

All Azure resources created for this project are organised inside a single Resource Group called MedicineReminder-RG. A Resource Group is simply a logical container in Azure that groups related cloud services together, making them easy to manage, monitor, and delete as a unit.

The resource group contains the following services:

- medicine-reminder-function-fq — the Function App running both serverless functions.
- MedicineReminderLogicApp — the Logic App acting as the daily scheduler.
- medicinereminderstorage — the Blob Storage account holding medicines.json.
- ASP-MedicineReminderRG-b9f0 — the App Service Plan (Consumption tier).

- Application Insights — automatic performance and error monitoring.
- func-performance-alert — an alert rule for unusual function behaviour.
- storage-availability-alert — an alert if Blob Storage availability drops below 90%.

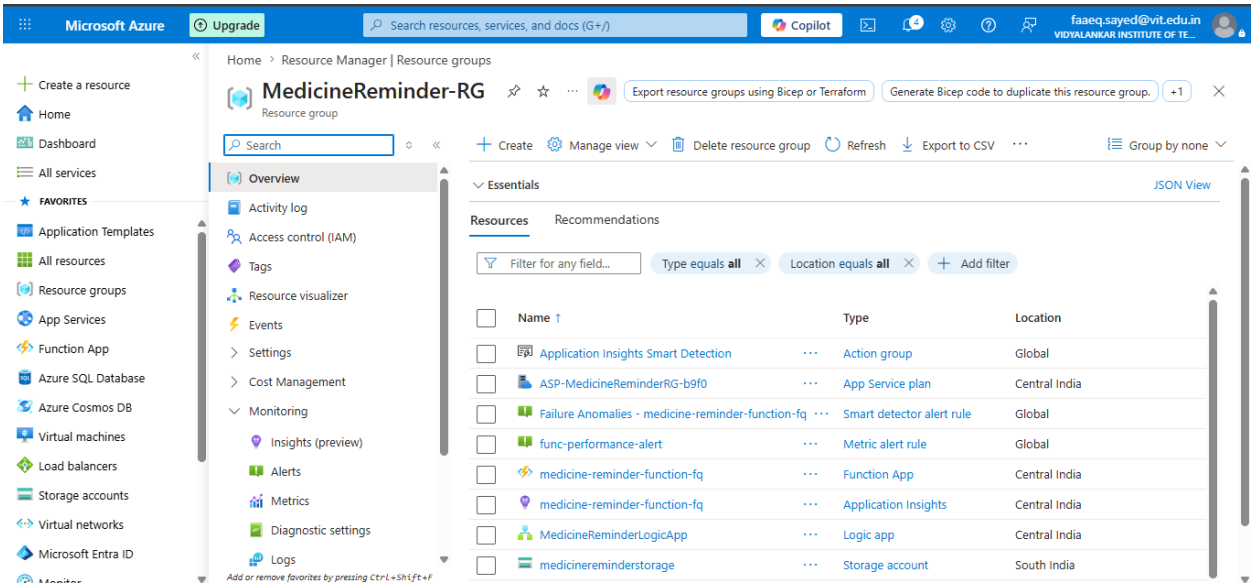


Figure 10: The MedicineReminder-RG resource group listing all Azure resources used in this project

8. Monitoring & Alerts → Built-in Safety Net

The project includes production-grade monitoring to ensure reliability. Two alert rules have been configured:

Storage Availability Alert

An alert named storage-availability-alert monitors the Blob Storage account. If the availability metric drops below 90%, Azure automatically notifies the team. This ensures that if something goes wrong with the data storage layer, the team is informed immediately rather than discovering the problem through missed emails.

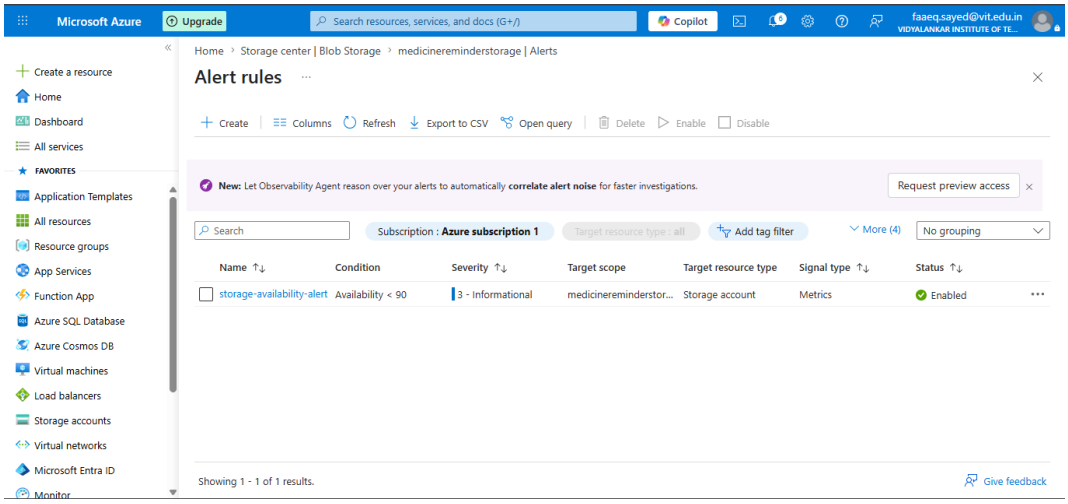
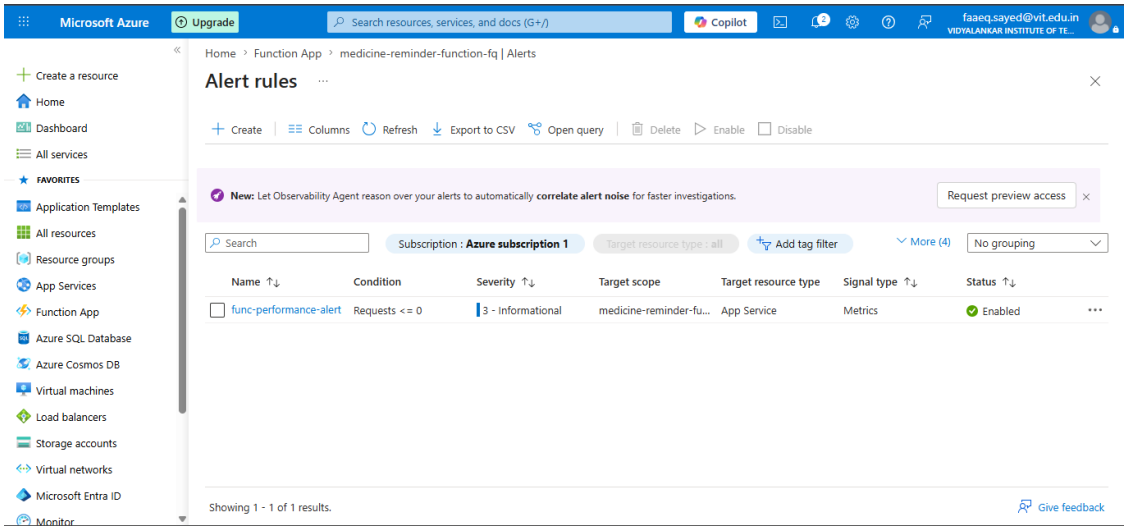


Figure 11: The storage-availability-alert configured on the Blob Storage account — set to fire if availability drops below 90%

Application Insights

The Function App is connected to Azure Application Insights, which automatically logs every function execution, captures errors, and provides dashboards to track performance over time. This gives the team complete visibility into how the system is behaving in production.



9. System Output — The Emails Patients Receive

The entire purpose of this system is to deliver timely, clear emails to patients. Here are the four types of emails the system sends:

9.1 Medicine Added Confirmation

Immediately after a patient submits the form and the medicine is saved successfully, an email is sent confirming all the details — the medicine name, scheduled time, and expiry date. This gives the patient confidence that the system has recorded their information correctly.

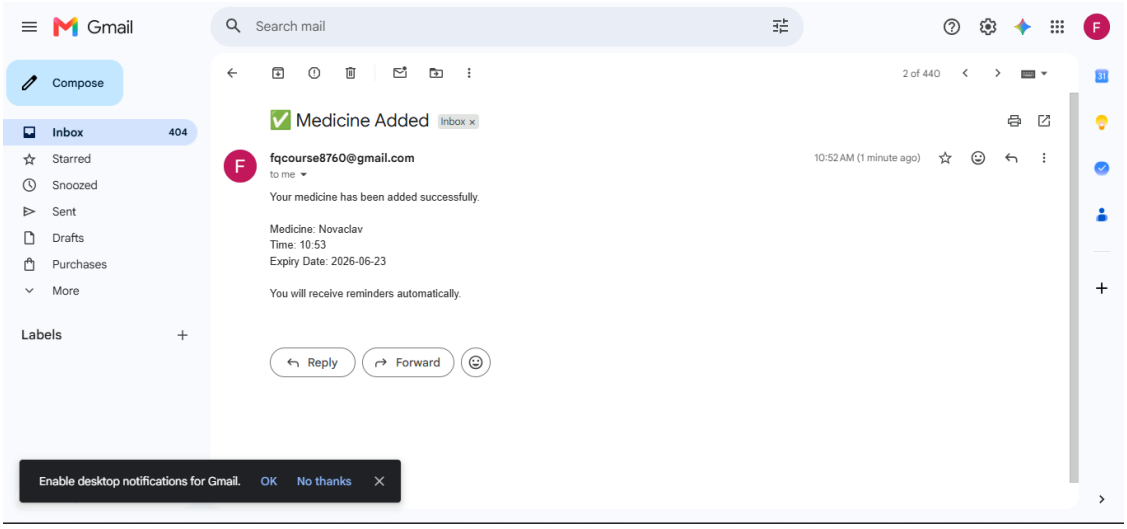
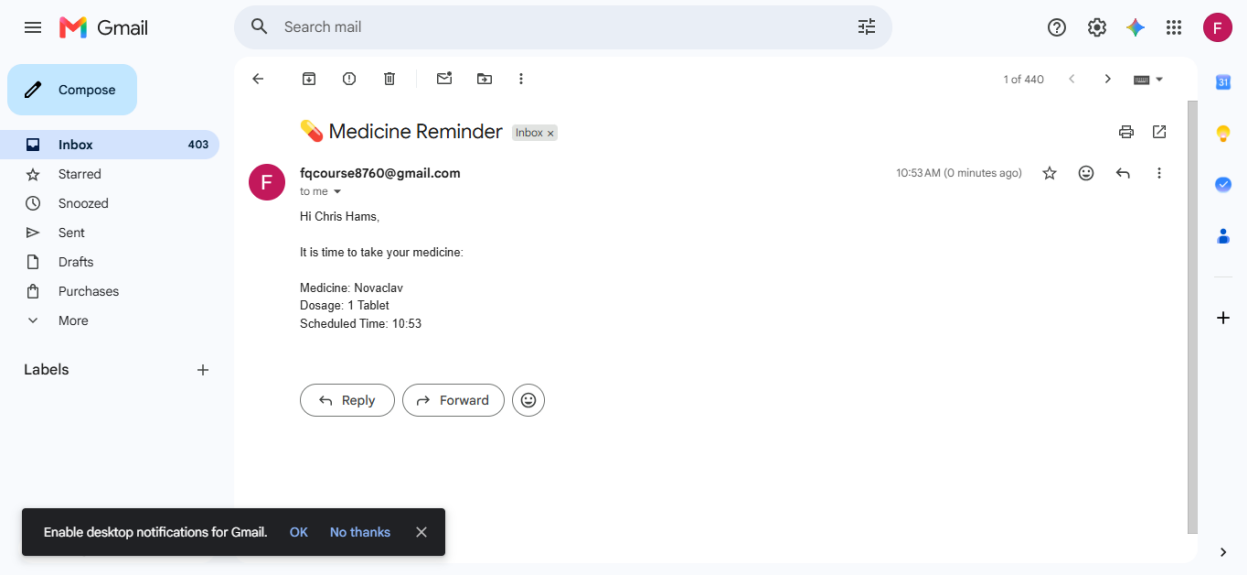


Figure 12: The "Medicine Added" confirmation email received by the patient, showing Novaclav registered at 10:52 AM with expiry on 23 June 2026

9.2 Daily Medicine Reminder

At the exact time the patient specified, an email arrives reminding them to take their medicine, including the medicine name, dosage, and sc



heduled time. This is the core feature of the project.

Figure 13: A live "Medicine Reminder" email delivered at 10:53 AM to patient Chris Hams, reminding them to take 1 Tablet of Novaclav

9.3 Medicine Expiring Tomorrow Warning

At 9:00 AM the morning before a medicine's expiry date, the patient receives a warning email asking them to arrange a replacement. This gives them a 24-hour window to act.

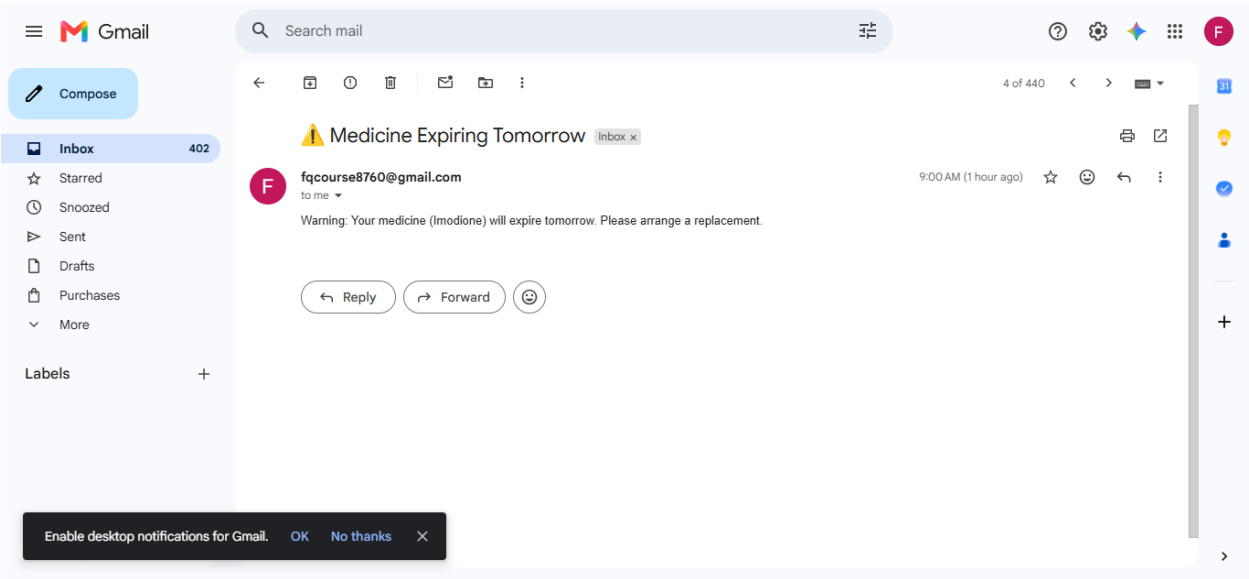


Figure 14: The "Medicine Expiring Tomorrow" warning email for Imodione, sent at 9:00 AM to prompt the patient to arrange a replacement

9.4 Medicine Expired Today Alert

On the day of expiry itself, if the patient's medicine has now expired, the system sends a clear, urgent alert advising them not to consume it.

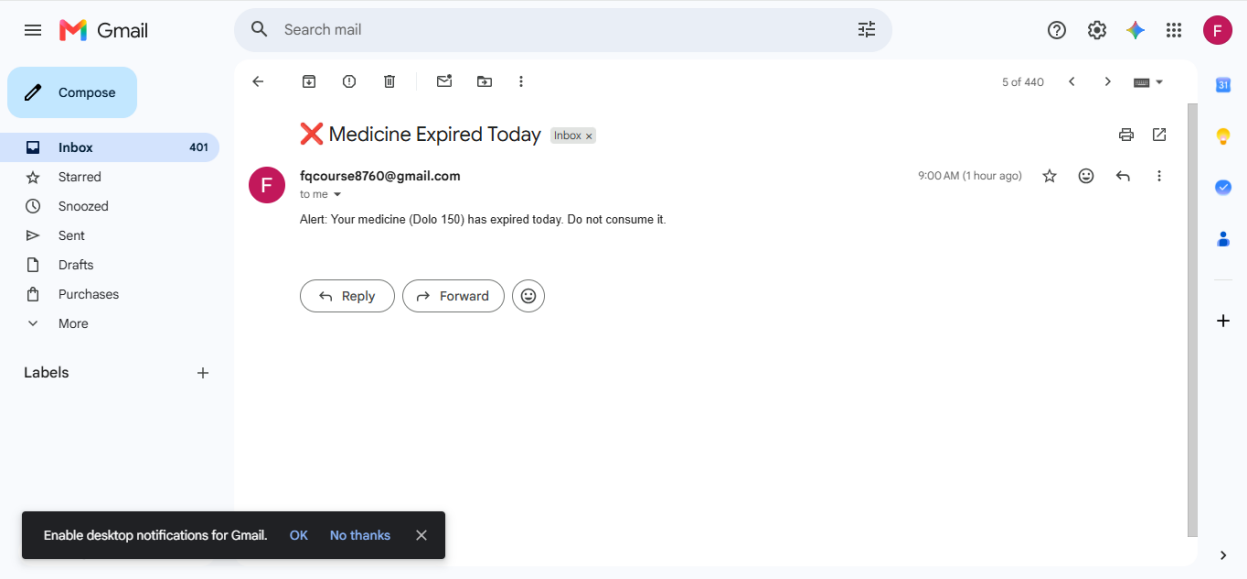


Figure 15: The "Medicine Expired Today" alert for Dolo 150, sent at 9:00 AM warning the patient not to consume the expired medicine

10. Project File Structure

The project codebase is organised into five key files, each with a distinct responsibility:

File	Purpose
`index.html`	The frontend web form captures patient name, email, medicine name, dosage, time, and dates.
`style.css`	Applies the dark-mode glassmorphic visual design to the web interface.
`app.js`	Handles the form submission on the client side, validates input, converts AM/PM time to 24-hour format, and sends data to the Azure Function via HTTPS.
`AddMedicine.js`	The Azure serverless function that receives patient data, saves it to Blob Storage, and sends the confirmation email.
`CheckMedicineReminders.js`	The core automation script reads all records, checks scheduled times, runs the 9 AM expiry scan, and sends the appropriate emails.

11. Complete Technology Stack

Category	Technology	Purpose
Frontend	HTML5 / CSS3 / JavaScript (ES6)	Patient-facing web form with glassmorphic design
Cloud Platform	Microsoft Azure	Hosts all serverless infrastructure
Serverless Functions	Azure Functions (Node.js v20)	API endpoints and timer-based automation
Data Storage	Azure Blob Storage	Stores patient records in medicines.json
Workflow Scheduler	Azure Logic Apps	Daily trigger to wake up functions
Email Delivery	Nodemailer + Gmail SMTP	Sends all patient email notifications
Version Control	Git + GitHub	Source code repository management
Monitoring	Azure Application Insights	Error tracking and performance dashboards

12. Conclusion

The Smart Medicine Reminder & Expiry Tracker System demonstrates how modern cloud infrastructure can be used to solve real, everyday healthcare problems without requiring expensive hardware or complex software maintenance.

By combining Azure Functions, Blob Storage, and Logic Apps into a cohesive, event-driven pipeline, the team built a system that is reliable, cost-efficient, and fully automated. The choice of the Azure Consumption Plan with Node.js driven both by budget constraints (no Flex Consumption Plan support on free trial accounts) and by the familiarity of JavaScript across the full stack resulted in a clean, maintainable codebase.

The anti-duplicate idempotency mechanism using date-string locks, rather than fragile boolean flags, shows thoughtful engineering that goes beyond a basic prototype. The addition of monitoring alerts and Application Insights brings the project closer to production-quality standards.

Most importantly, the system works exactly as intended: a patient enters their details once, and from that moment on, the cloud takes care of the rest sending dose reminders at the right time, warning about upcoming expiries, and alerting patients the moment a medicine crosses its expiry date.